Sorbonne Université

Master of Computer Science Computer Networks: Internet, Cybersecurity, Cloud and Automation (RES)

IOB

ChainTrack: A Blockchain-Based Technical Framework for Immutable Supply Chain Provenance

Conducted by:

ABDELWAFI Ibtissam

BOUSKRI Mohamed

Supervised by:

Prof. Maria Potop-Butucaru

Academic Year 2025/2026

Table of contents

Table of contents 2
Abstract 4
1 Introduction 5
1.1 The Current State of Global Supply Chains 5
1.2 The "Internet of Blockchain" (IoB) Paradigm 5
2 Problem Statement 5
2.1 Information Asymmetry 5
2.2 Counterfeiting and Fraud 5
2.3 Lack of Real-Time Visibility 6
3 Literature Review 6
3.1 Evolution of Supply Chain Management (SCM) 6
3.2 Centralized vs. Decentralized Ledgers 6
3.3 Existing Blockchain Solutions in Industry 6
4 Core Blockchain Concepts and the IOB Framework 6
4.1 Distributed Ledger Technology (DLT) 6
4.2 Smart Contracts: The Programmable Trust Layer 7
4.3 Consensus Mechanisms 7
4.4 Tokenization of Physical Assets 7
5 System Architecture 7
5.1 High-Level Overview 7
5.2 The Technical Stack (Hardhat, Ethers.js, React) 7
5.3 Data Flow and Interaction Model 7
6 Technical Implementation: The Smart Contract Layer 8
6.1 Core Data Structures 8
6.2 Functionality Deep-Dive 9
6.3 On-Chain Metadata and "Rich Details" 9
6.3.1 The Metadata Schema 9
6.3.2 Immutability of Records 9
6.4 Implementation Snippets 9
6.5 Optimization and Gas Management 11

7 Technical Implementation: Frontend Orchestration 11
7.1 Bridging Web2 and Web3 11
7.2 State Management and Reactive UI 11
7.3 User Interface (UI) Design and Functional Systems 11
7.4 MetaMask and Wallet Integration 12
8 Security Analysis and Governance 12
8.1 Access Control and Role-Based Permissions 12
8.2 Immutability and Auditability 12
8.3 Potential Attack Vectors 13
9 Functional Guide: How the Application Works 13
9.1 Phase 1: Product Tokenization (Creation) 13
9.2 Phase 2: Journey Tracking and Location Updates 14
9.3 Phase 3: Secure Ownership Transfer (Multi-Account Proof) 15
9.4 Phase 4: Lifecycle Decommissioning (Archiving) 18
9.4.1 Authorization of Lifecycle Termination 18
9.4.2 Archival Confirmation 19
9.4.3 Final State Visualization 20
10 Case Studies and Potential Applications 20
11 The IoB Perspective: Future Integrations 21
11.1 IoT and Autonomous Sensor Feedback 21
11.2 Zero-Knowledge Proofs (ZK-Proofs) 21
11.3 Cross-Chain Interoperability 21
12 Socio-Economic and Environmental Impact 21
12.1 Transparency and Ethical Sourcing 21
12.2 Reducing Administrative Overhead 21
12.3 Sustainability and Carbon Tracking 21
13 Quick Testing via Remix 21
Conclusion 23
References 24
Annex A – Smart Contract Implementation (Solidity) 25

Abstract

Global supply chains have become increasingly complex due to globalization, outsourcing, and the multiplication of intermediaries involved in the lifecycle of a single product. While this complexity enables efficiency and scalability, it also introduces significant challenges related to transparency, traceability, fraud, and trust between stakeholders who do not necessarily share the same objectives or information systems. Traditional supply chain information systems are largely centralized, fragmented, and opaque, making them ill-suited to address issues such as counterfeiting, unethical sourcing, and lack of real-time visibility.

This report presents ChainTrack, a decentralized application (dApp) designed to provide immutable, verifiable, and transparent product provenance using blockchain technology, within the conceptual framework of the Internet of Blockchain. ChainTrack models physical goods as digital twins on an Ethereum-compatible blockchain, ensuring that every significant lifecycle event; creation, transfer, update, and decommissioning; is permanently recorded and auditable. The system relies on smart contracts written in Solidity, a React-based frontend, and an interaction layer built with Ethers.js to bridge Web2 and Web3 environments.

Beyond implementation, this report situates ChainTrack within existing academic and industrial work on blockchain-based supply chains, analyzes its security and governance properties, and explores future extensions such as IoT integration, zero-knowledge proofs, and cross-chain interoperability. The project demonstrates that blockchain, when carefully architected and combined with user-centric design, can serve as a foundational trust layer for next-generation supply chain systems.

1 Introduction

The rapid expansion of global trade networks has fundamentally transformed how goods are produced, transported, and consumed. Products routinely traverse multiple countries, legal jurisdictions, and organizational boundaries before reaching the end consumer. While this globalization has increased efficiency and reduced costs, it has simultaneously amplified structural weaknesses in existing supply chain information systems. Issues such as limited transparency, fragmented data ownership, and lack of real-time trust remain persistent challenges.

1.1 The Current State of Global Supply Chains

Most contemporary supply chains rely on centralized information systems, typically implemented through proprietary databases or Enterprise Resource Planning (ERP) platforms. Although these systems are effective for internal coordination, they are poorly suited for cross-organizational collaboration. Each stakeholder maintains its own version of the truth, resulting in data silos and inconsistencies. When disruptions occur such as product recalls, shipment delays, or fraud investigations, reconstructing an accurate history often requires manual reconciliation of heterogeneous records, leading to delays and increased operational costs.

1.2 The "Internet of Blockchain" (IoB) Paradigm

The Internet of Blockchain (IoB) paradigm proposes a decentralized alternative to traditional trust models. Rather than relying on a central authority to validate transactions and records, IoB leverages cryptographic consensus mechanisms to establish trust across distributed participants. In the context of supply chains, IoB enables physical goods to be represented as digital assets whose state and history are shared across a tamper-resistant network. This paradigm shift transforms trust from an organizational property into a protocol-enforced guarantee.

2 Problem Statement

Despite decades of optimization, modern supply chains continue to suffer fundamental trust and transparency limitations. These shortcomings not only increase operational inefficiencies but also expose consumers and regulators to significant risks.

2.1 Information Asymmetry

Information asymmetry is inherent to most supply chains, where upstream actors possess detailed knowledge about sourcing and production, while downstream stakeholders (particularly consumers) have limited visibility. This imbalance enables unverifiable claims related to sustainability, ethical sourcing, or quality standards, undermining consumer trust and regulatory oversight.

2.2 Counterfeiting and Fraud

Counterfeiting represents a critical challenge across multiple industries, including luxury goods, pharmaceuticals, and food supply chains. Existing authentication mechanisms often depend on centralized databases or static identifiers, which can be replicated or manipulated.

Without a shared and immutable provenance record, verifying authenticity remains costly and unreliable.

2.3 Lack of Real-Time Visibility

Traditional supply chain monitoring systems are predominantly reactive. Updates are frequently logged after events occur, reducing the ability to detect anomalies or intervene proactively. This lack of real-time visibility limits optimization opportunities and increases exposure to operational and reputational risks.

3 Literature Review

3.1 Evolution of Supply Chain Management (SCM)

Supply Chain Management has evolved from basic logistics coordination to highly integrated digital platforms. However, most innovations have focused on efficiency rather than trust. ERP systems optimize internal workflows but do not address the inter-organizational trust deficit that characterizes global supply networks.

3.2 Centralized vs. Decentralized Ledgers

A centralized ledger is efficient but requires a "Trusted Third Party." In many global scenarios, such a party does not exist or is too expensive to maintain across borders. Decentralized ledgers (blockchains) solve the "Byzantine Generals' Problem," allowing parties who do not trust each other to agree on a common state of data without a central mediator.

3.3 Existing Blockchain Solutions in Industry

The feasibility of blockchain in logistics is no longer theoretical. Major global initiatives have demonstrated significant returns on investment:

- **IBM Food Trust**: By linking producers, processors, and retailers (like Walmart), this consortium reduced the time to trace mangoes from the farm to the store from 7 days to 2.2 seconds.
- **TradeLens (Maersk/IBM)**: This platform digitized global shipping documents (Bills of Lading), drastically reducing the administrative "paper trail" that historically accounted for up to 20% of the cost of physical transport.
- **Provenance.org**: Focused on consumer-facing transparency, Provenance allows luxury and fashion brands to prove sustainability claims through verifiable material tracking.

ChainTrack builds on these precedents by applying the "Internet of Blockchain" (IoB) framework. While enterprise solutions are often complex and costly, ChainTrack provides a lightweight, modular alternative that demonstrates these core principles, immutability and cryptographic provenance, for small-to-medium scale operations.

4 Core Blockchain Concepts and the IOB Framework

4.1 Distributed Ledger Technology (DLT)

ChainTrack utilizes a Peer-to-Peer (P2P) network where every node maintains a copy of the ledger. This redundancy ensures high availability and prevents a single point of failure.

4.2 Smart Contracts: The Programmable Trust Layer

Smart contracts are self-executing contracts with the terms of the agreement between buyer and seller being directly written into lines of code. In ChainTrack, the smart contract acts as an automated auditor that only allows valid state transitions.

4.3 Consensus Mechanisms

While our implementation runs on a local Hardhat network for development, in a live IoB environment, consensus mechanisms like Proof of Stake (PoS) ensure that transactions are validated without the massive energy consumption seen in older Proof of Work (PoW) systems.

4.4 Tokenization of Physical Assets

Tokenization is the process of representing a physical good as a digital token on the blockchain. By linking a unique digital ID to a physical product, we create a "digital twin" that tracks the item's state and ownership in real-time.

5 System Architecture

5.1 High-Level Overview

The system architecture follows a three-tier model:

1. **Blockchain Layer**: The "Source of Truth" where the SupplyChain smart contract resides.
2. **Interaction Layer**: The middleware consisting of Ethers.js that facilitates communication between the browser and the blockchain.
3. **Application Layer**: The React-based frontend provides the user experience.

5.2 The Technical Stack (Hardhat, Ethers.js, React)

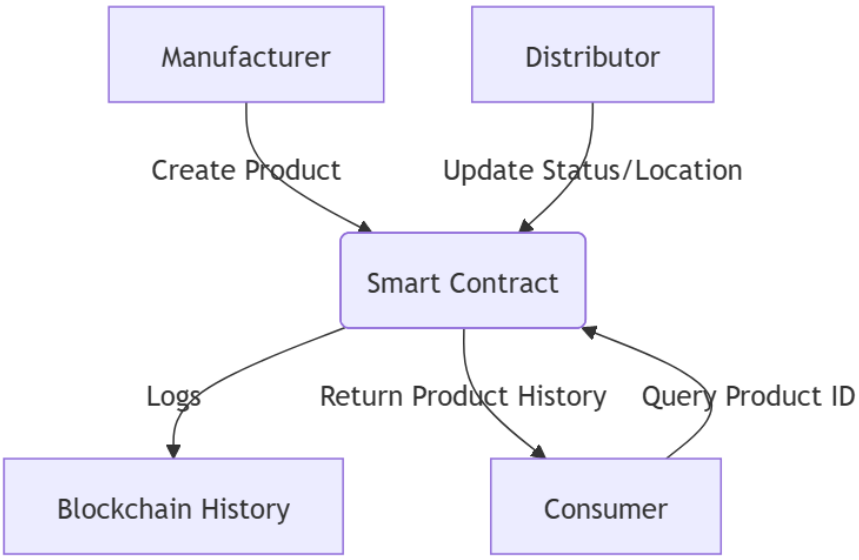
- **Hardhat**: A development environment to compile, deploy, and test Ethereum software. It provides a local EVM (Ethereum Virtual Machine).
- **Solidity**: The high-level language used for writing smart contracts.
- **Vite + React**: A fast, modern development server and UI library for building reactive interfaces.
- **Ethers.js**: Library for blockchain interactions.

5.3 Data Flow and Interaction Model

The interaction model is designed to maintain a synchronized state across the decentralized network while providing a responsive user experience.

1. **State Initiation**: The frontend (React) captures user input and signs it using the private keys held in MetaMask.
2. **Transaction Execution**: Ethers.js broadcasts the signed transaction to the local Hardhat node (or public Ethereum node).

3. **On-Chain Mutation**: The SupplyChain smart contract validates the transaction, updates the internal mapping, and appends a new History node to the product's array.
4. **Asynchronous Feedback**: Once the block is mined, the contract emits an event (e.g., ProductCreated). The frontend listens for these events to provide immediate confirmation to the user.
5. **Query Path**: For product tracking, the frontend makes a read-only call to getProductHistory. Since this does not mutate state, it is executed locally by the node without a gas cost, ensuring high-speed data retrieval for consumers.



6 Technical Implementation: The Smart Contract Layer

6.1 Core Data Structures

The contract is centered around two primary data structures: Product and History.

The use of a struct for History allows us to capture rich metadata for every transaction.

```
struct History {
   string action;
   string details;
   address actor;
   uint256 timestamp;
}
struct Product {
   uint256 id;
   string name;
   string description;
   string origin;
   address currentOwner;
   bool isActive;
   History[] history;
}
```

6.2 Functionality Deep-Dive

- **createProduct**: Initializes a new entry. The ID is auto incremented to ensure uniqueness.
- **transferProduct**: Changes the currentOwner address. Crucially, it requires msg.sender == product.currentOwner, enforcing that only the rightful holder can pass the asset.
- **updateProduct**: Allows the current owner to append logs without changing ownership, useful for transit updates or quality checks.
- **deactivateProduct**: An administrative/ownership function that marks a product as isActive = false. This represents the end of the product's operational lifecycle (e.g., consumption, destruction, or decommissioning).

6.3 On-Chain Metadata and "Rich Details"

A core innovation of ChainTrack is its handling of metadata. Unlike traditional databases that might overwrite status fields, ChainTrack utilizes a "cumulative metadata" model.

6.3.1 The Metadata Schema

The system captures three types of metadata:

- **Identity Metadata**: Immutable attributes defined at creation (name, origin).
- **State Metadata**: Mutable fields reflecting the status (currentOwner, isActive).
- **Auditable History Metadata**: Every state change appends a History object. This includes the actor (address), timestamp (Unix time), and the details field, a flexible string allowing stakeholders to input specific context such as batch numbers, sensor readings, or logistics notes.

6.3.2 Immutability of Records

Because each metadata update is pushed to an array rather than replaced, auditors can "replay" the entire journey of a product, ensuring no stakeholder has tampered earlier logs.

6.4 Implementation Snippets

Below are key segments of the smart contract implementation that illustrate the architectural choices mentioned above.

Product Creation with History Tracking

```
function createProduct(string memory _name, string memory _description, string memory _origin) public returns (uint256) {
   _productIds++;
   uint256 newId = _productIds;
   Product storage newProduct = products[newId];
   newProduct.id = newId;
   newProduct.name = _name;
   newProduct.description = _description;
   newProduct.origin = _origin;
```

```
    newProduct.currentOwner = msg.sender;
    newProduct.isActive = true;
    newProduct.history.push(History({
        action: "Created",
        details: string(abi.encodePacked("Origin: ", _origin)),
        actor: msg.sender,
        timestamp: block.timestamp
    }));
    emit ProductCreated(newId, _name, msg.sender);
    return newId;
}
```

Secure Ownership Transfer

```
function transferProduct(uint256 _productId, address _newOwner, string memory
_details) public {
    Product storage product = products[_productId];
    require(product.isActive, "Product is not active");
    require(msg.sender == product.currentOwner, "Only owner can transfer");
    require(_newOwner != address(0), "Invalid new owner");
    address previousOwner = product.currentOwner;
    product.currentOwner = _newOwner;
    product.history.push(History({
        action: "Transferred",
        details: _details,
        actor: msg.sender,
        timestamp: block.timestamp
    }));
    emit ProductTransferred(_productId, previousOwner, _newOwner, _details);
}
```

Product update

```
function updateProduct(uint256 _productId, string memory _details) public {
        Product storage product = products[_productId];
        require(product.isActive, "Product is not active");
        // Allow owner to update status.
        require(msg.sender == product.currentOwner, "Only owner can update");

        product.history.push(History({
            action: "Updated",
            details: _details,
            actor: msg.sender,
            timestamp: block.timestamp
        }));

        emit ProductUpdated(_productId, _details, msg.sender);
    }
```

Lifecycle Deactivation

```
function deactivateProduct(uint256 _productId) public {
   Product storage product = products[_productId];
   require(product.isActive, "Product is already inactive");
   require(msg.sender == product.currentOwner, "Only owner can deactivate");
   product.isActive = false;
   product.history.push(History({
       action: "Deactivated",
       details: "Product marked as inactive/archived",
       actor: msg.sender,
       timestamp: block.timestamp
   }));
   emit ProductDeactivated(_productId, msg.sender);
}
```

6.5 Optimization and Gas Management

By setting the initialBaseFeePerGas to 0 in the Hardhat network configuration, the system facilitates rapid development and testing without the friction of acquiring testnet ether, while still adhering to the standard EVM transaction lifecycle.

7 Technical Implementation: Frontend Orchestration

7.1 Bridging Web2 and Web3

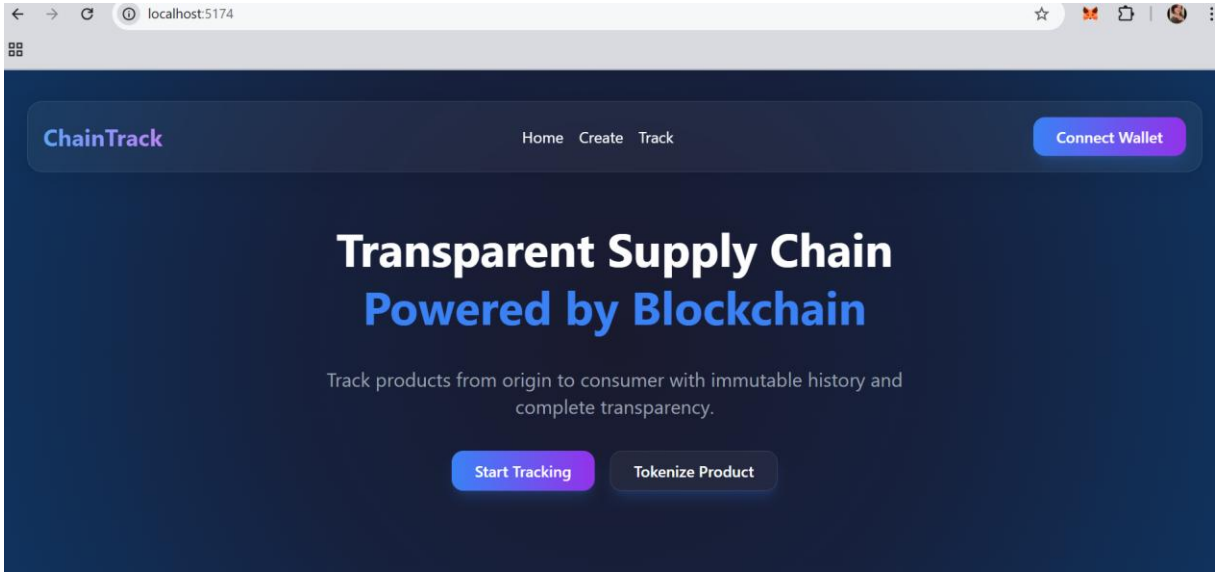
The frontend uses the ethers.BrowserProvider to tap into the user's browser-injected Ethereum provider (MetaMask). This allows for a "passwordless" experience where identity is managed by cryptographic keys.

7.2 State Management and Reactive UI

We utilize React's useState and useEffect to synchronize the UI with the blockchain state. Given that blockchain transactions are asynchronous, the UI implements intensive loading states and "optimistic" feedback to maintain a smooth UX.

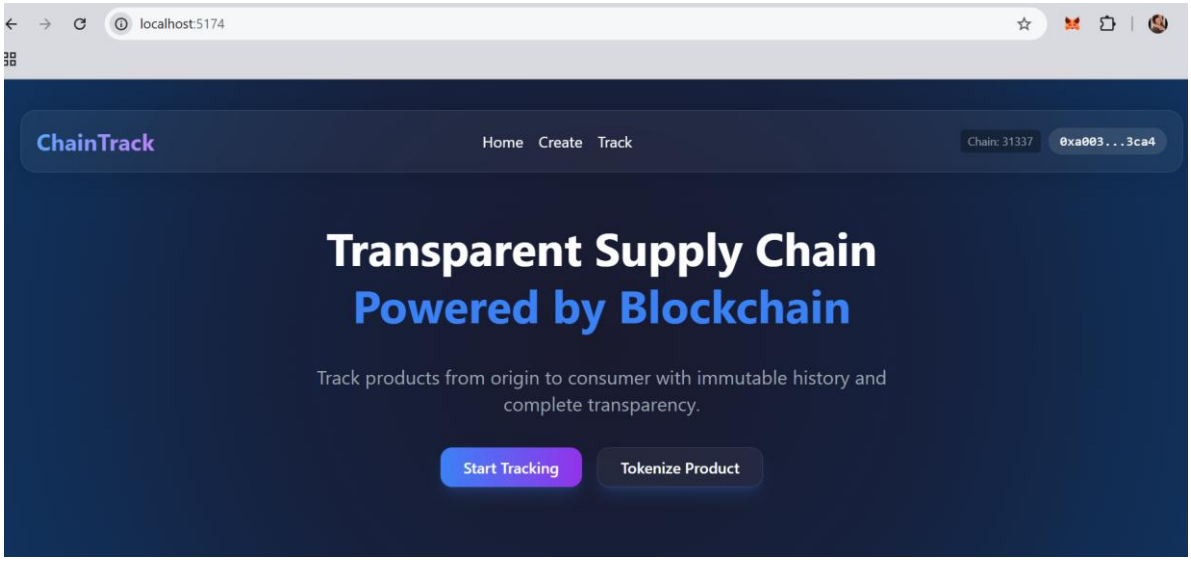
7.3 User Interface (UI) Design and Functional Systems

The interface is designed to prioritize data clarity and operational transparency. Key features include high-contrast status indicators that translate raw blockchain states (like isActive booleans) into human-readable badges. The layout is optimized for quick navigation between creation and tracking modes, ensuring that supply chain operators can manage assets with minimal cognitive load.



7.4 MetaMask and Wallet Integration

The application utilizes MetaMask as the primary gateway for user authentication and transaction authorization. This integration allows for a seamless transition from a "Guest" state to a "Connected" state, where the user's public address becomes their identity within the supply chain.



8 Security Analysis and Governance

8.1 Access Control and Role-Based Permissions

The implementation uses a strict ownership check. In more complex versions, this could be expanded using the OpenZeppelin AccessControl library to manage different roles like "Verifier," "Transport," and "Manufacturer."

8.2 Immutability and Auditability

Once a record is written to the blockchain, it cannot be changed. This is achieved by the fundamental structure of the blockchain, where each block contains a cryptographic hash of the previous block. Any attempt to modify history would invalidate all subsequent blocks.

8.3 Potential Attack Vectors

While the blockchain layer is secure, potential vulnerabilities exist at the interface level, such as "Phishing" or "Front-Running." Mitigations include clear transaction previews in MetaMask and robust error handling in the application layer.

9 Functional Guide: How the Application Works

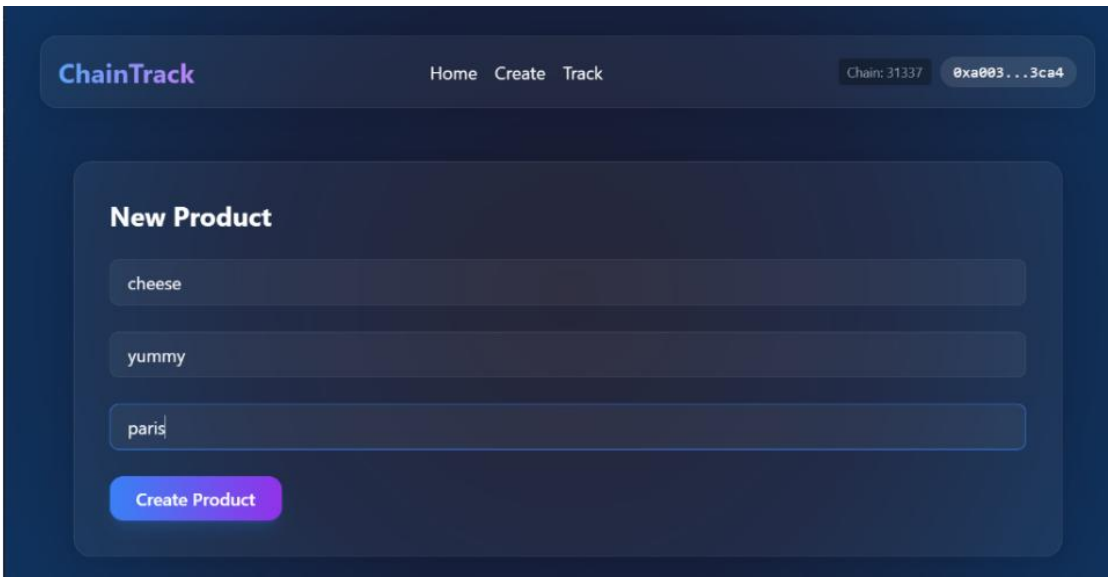
9.1 Phase 1: Product Tokenization (Creation)

The lifecycle starts with the Creation phase, where a physical item is digitally recorded on the blockchain for the first time. In this stage, the manufacturer (the first stakeholder in the chain) registers the product's fundamental metadata. By filling out the registration form, the user defines the product name, a descriptive summary of its properties, and its specific geographical origin.

When the user triggers the "Create Product" action, the frontend calls the createProduct function in the smart contract. This operation requires a cryptographic signature via MetaMask to authorize the state change. The contract then increments a private counter to assign a unique, immutable Product ID and generates a new Product struct. The initial "Created" event is pushed to the product's on-chain history array, capturing the timestamp and the manufacturer's wallet address. This creates a "Digital Twin" that serves as the permanent source of truth for the physical item's provenance.

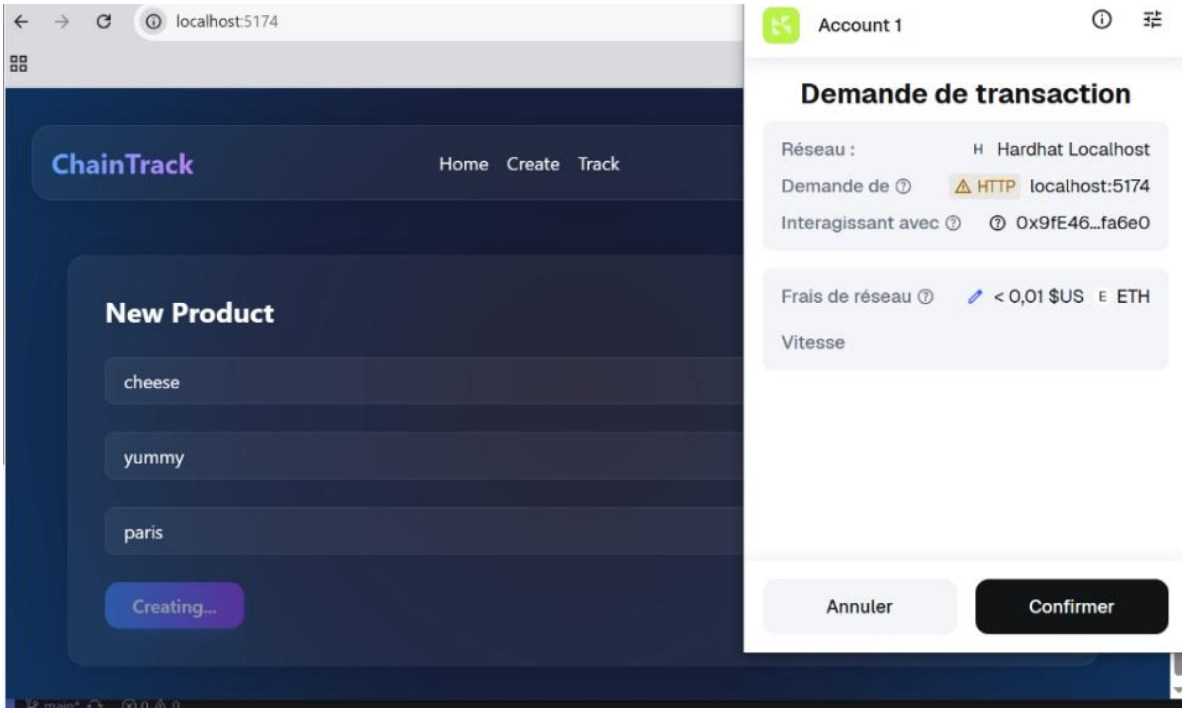
Step 1: Input Product Details

The user fills out the registration form within the "Create" interface.



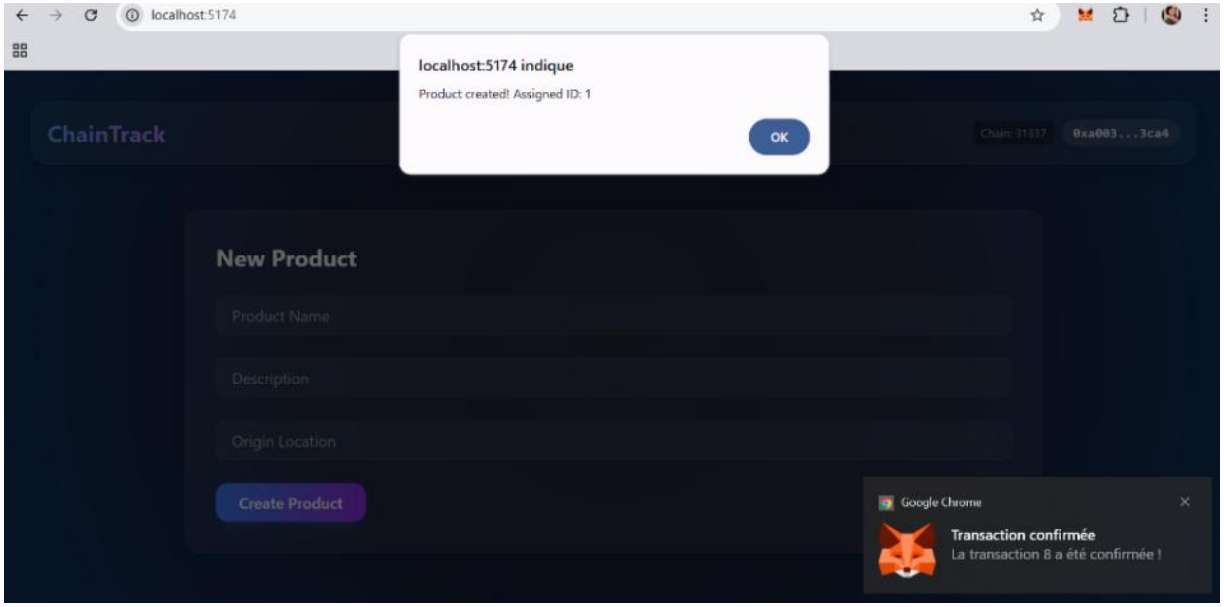
Step 2: Cryptographic Authorization

MetaMask triggers a transaction signing request, requiring the user to authorize the creation of the digital twin on chain.



Step 3: Deployment Confirmation

A system alert confirms the successful creation and displays the unique, auto-incremented Product ID (e.g., ID 1).

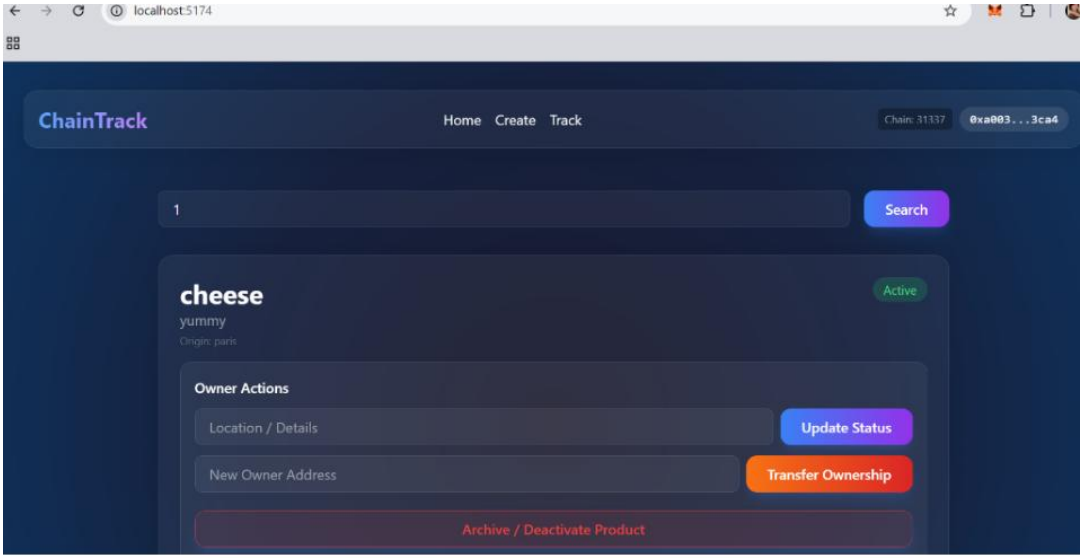


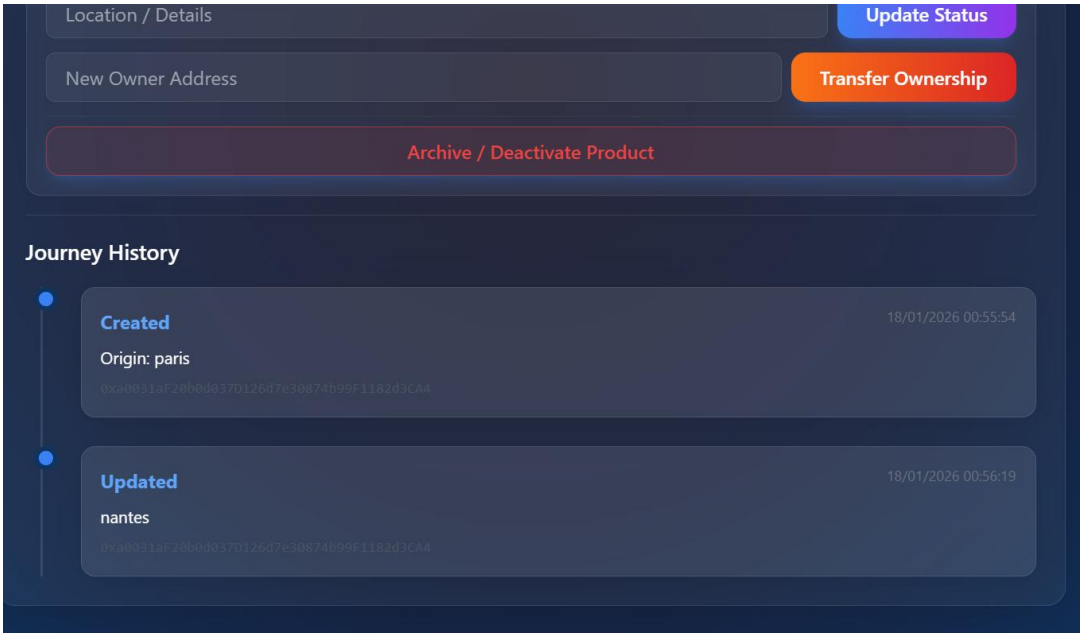
9.2 Phase 2: Journey Tracking and Location Updates

Once a product is live, it moves through various logistical nodes (warehouses, shipping ports, distribution centers). In the Tracking & Update phase, current possessors of the item can log into handling events or location changes without altering the legal ownership of the asset.

By entering the Product ID in the "Track" interface, any viewer can see the current status. However, only the authorized owner has the permission to call the updateProduct function. Each update, such as arrival in a new cityappends on a new History entry containing the

handler's address and the specific details of the update. This provides real-time visibility into the supply chain, allowing auditors and consumers to verify exactly where the product has been and how it was handled at every step of its journey.



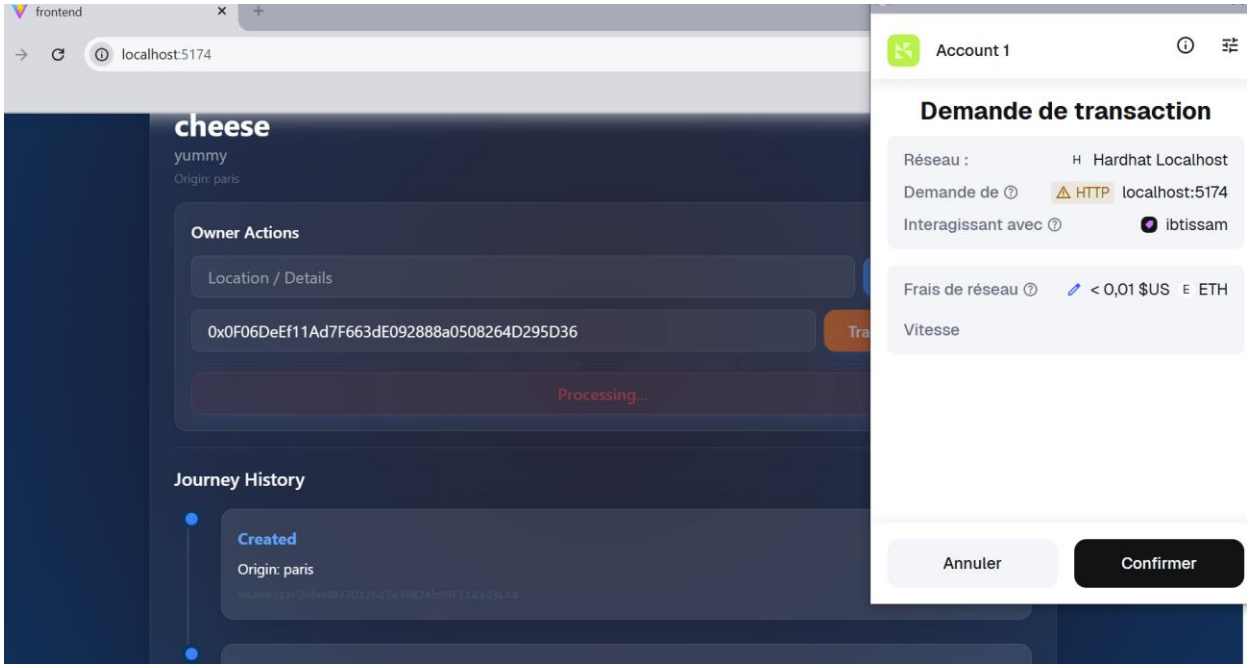


9.3 Phase 3: Secure Ownership Transfer (Multi-Account Proof)

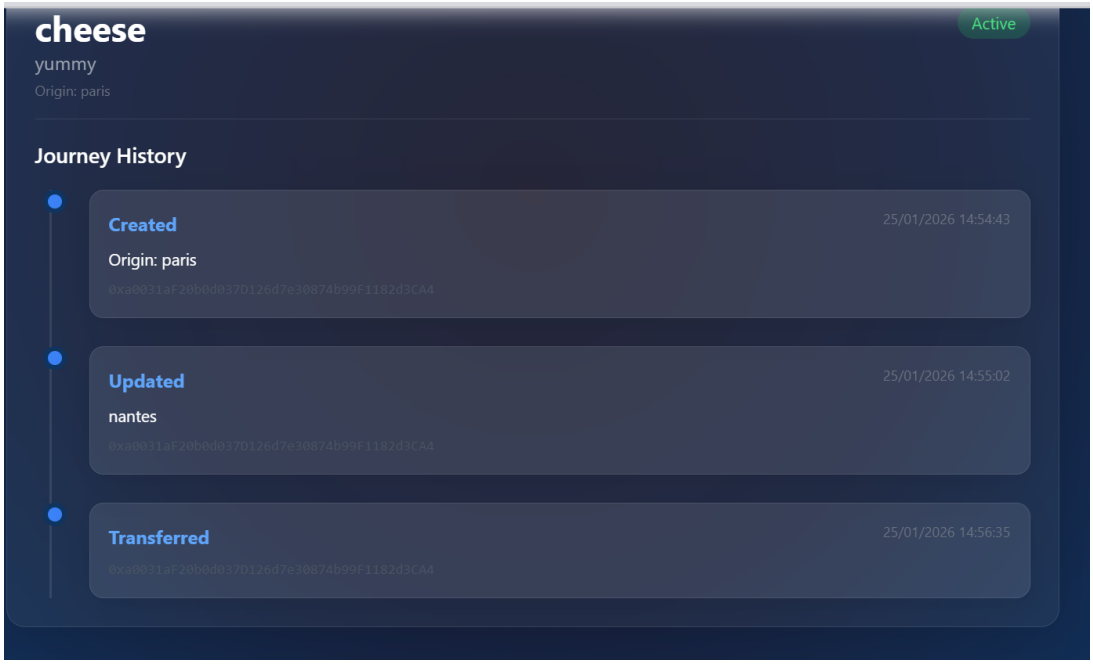
The Transfer phase is the most critical demonstration of the dApp's security and decentralized nature. This phase represents the handoff of the physical product between two distinct legal entities. To validate this, we simulated a handoff between Account 1 (Ibtissam), representing the Manufacturer, and Account 2 (Mohamed), representing the Lead Distributor.

The process is highly secure: the current owner (Ibtissam) must explicitly enter the public wallet address of the recipient (Mohamed) and sign a blockchain transaction. The smart contract validates that Ibtissam is indeed the rightful owner before updating the currentOwner field to Mohamed's address. This "Trustless" mechanism ensures that ownership can only be passed by the authorized holder, preventing any unauthorized claims. Below are the visual

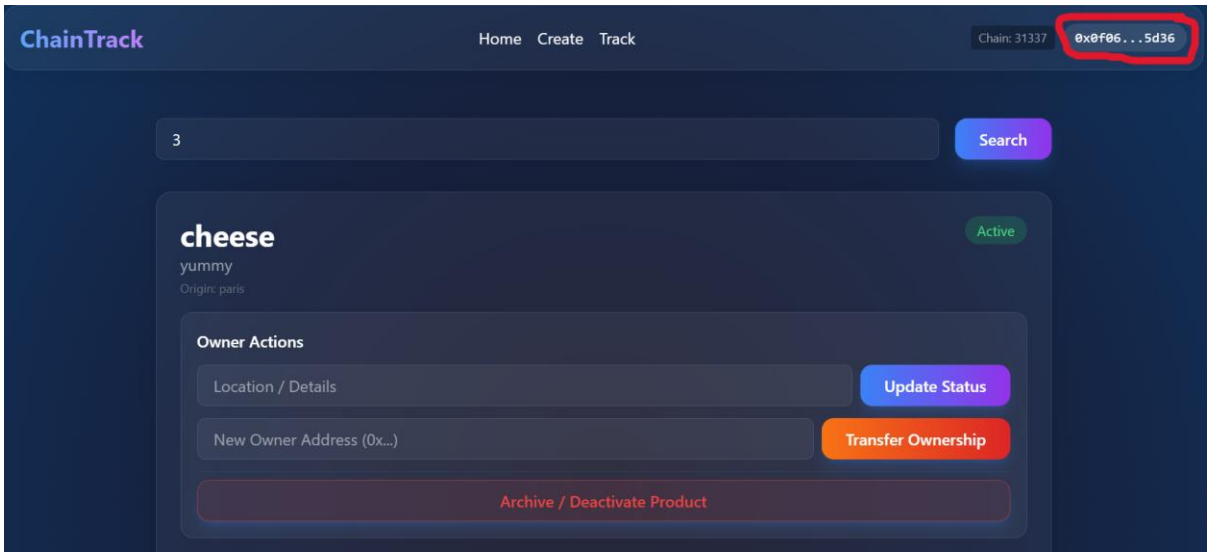
records of this specific verification process, showcasing the MetaMask authorization and the resulting immutable journey history that records the transition from Account 1 to Account 2.

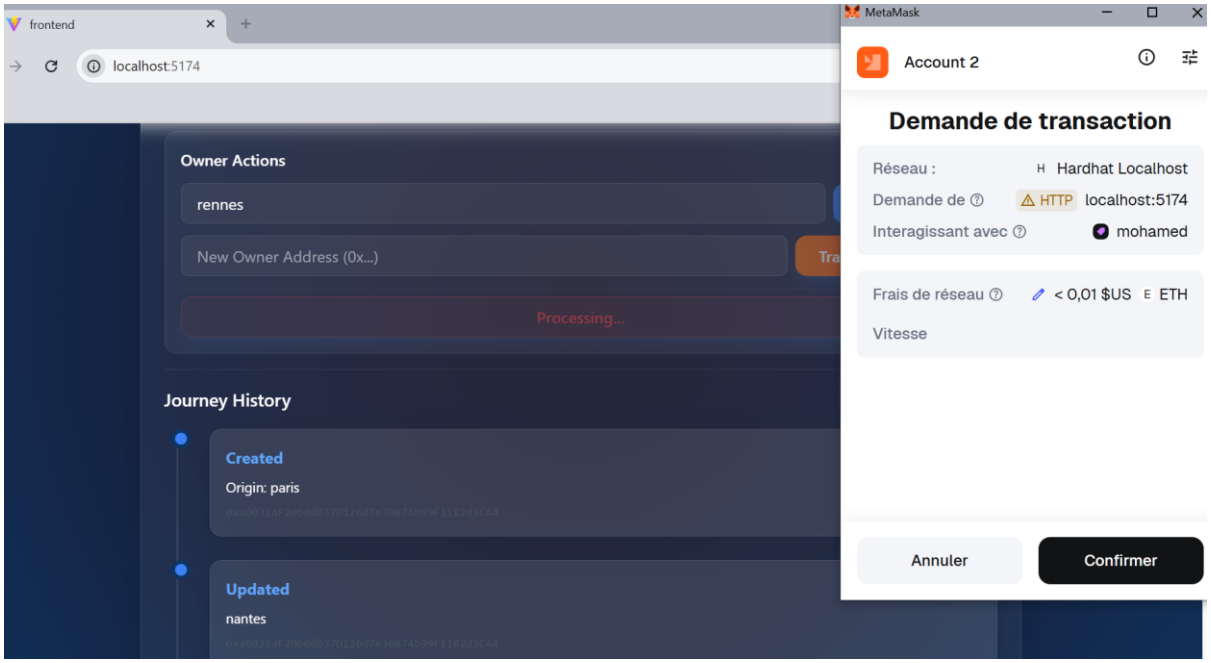


Account 1 (Ibtissam) authorizing the transfer to Mohamed and no longer have right to modify:

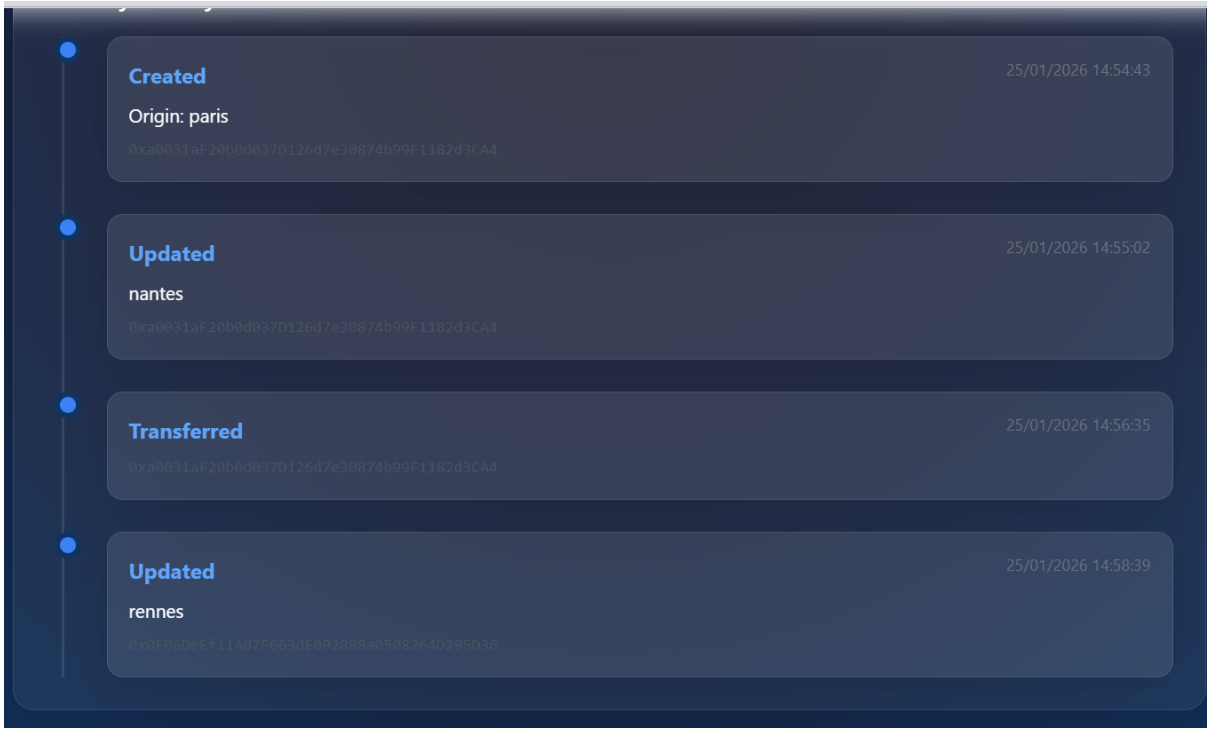


Now we connect from account 'Mohamed' and signing a subsequent status update in Rennes





Tracking UI: Product details for the 'Cheese' asset



9.4 Phase 4: Lifecycle Decommissioning (Archiving)

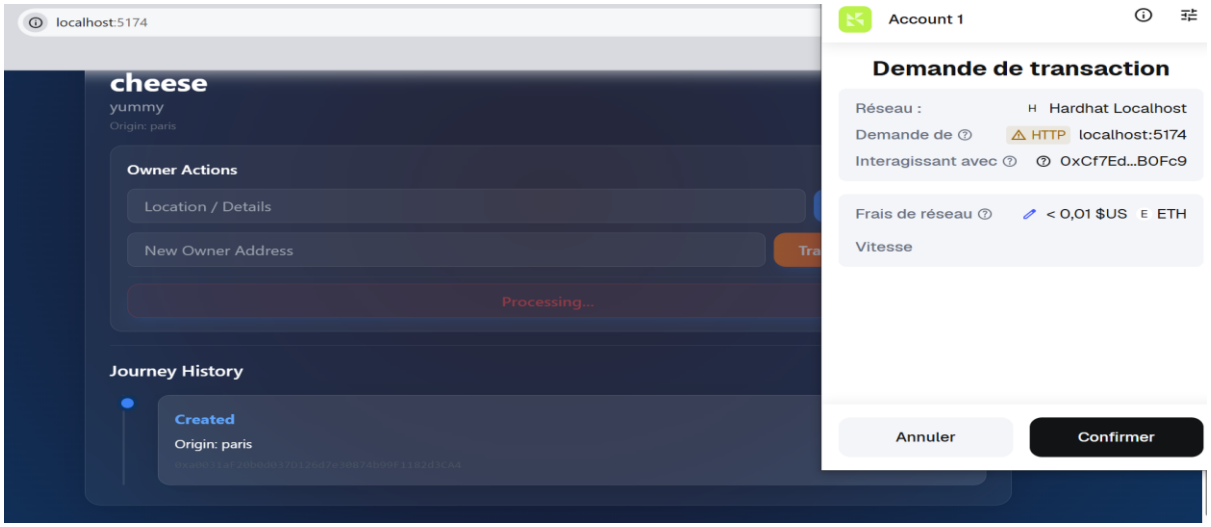
The final stage of the product lifecycle is Deactivation, occurring when the product reaches its endpoint (it is sold to a consumer, consumed, or decommissioned). The final owner triggers the deactivateProduct function, which sets the isActive flag to false in the smart contract.

This act places a "Cryptographic Seal" on the record. Once deactivated, the product's digital twin can no longer be updated or transferred, preventing fraudulent attempts to re-use the token for counterfeit items.

However, the entire history remains permanently stored on the blockchain, allowing anyone in the future to search for that ID and verify the authentic journey it took from Paris to its final destination. The UI displays an "Archived" badge, signaling that the asset's active life in the supply chain is complete.

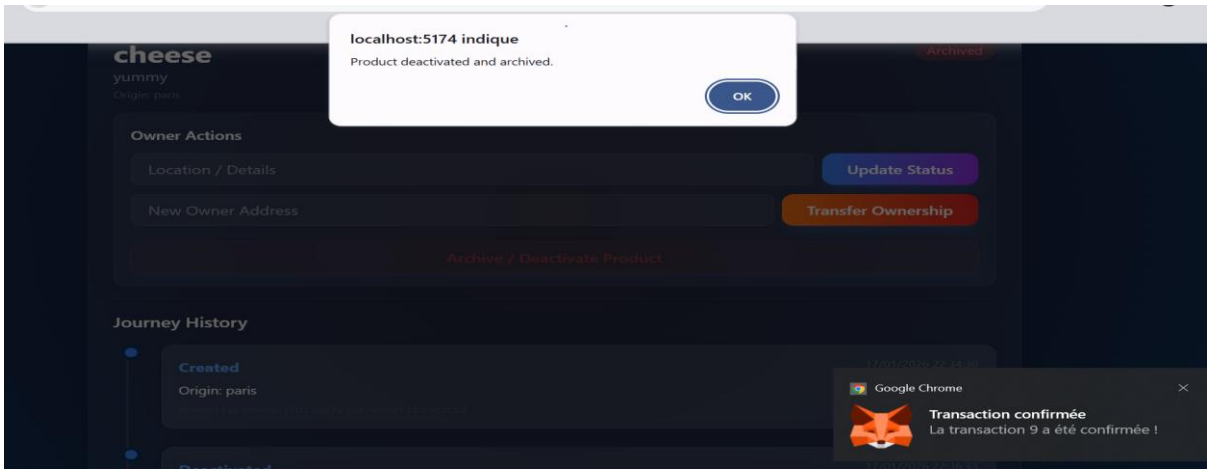
9.4.1 Authorization of Lifecycle Termination

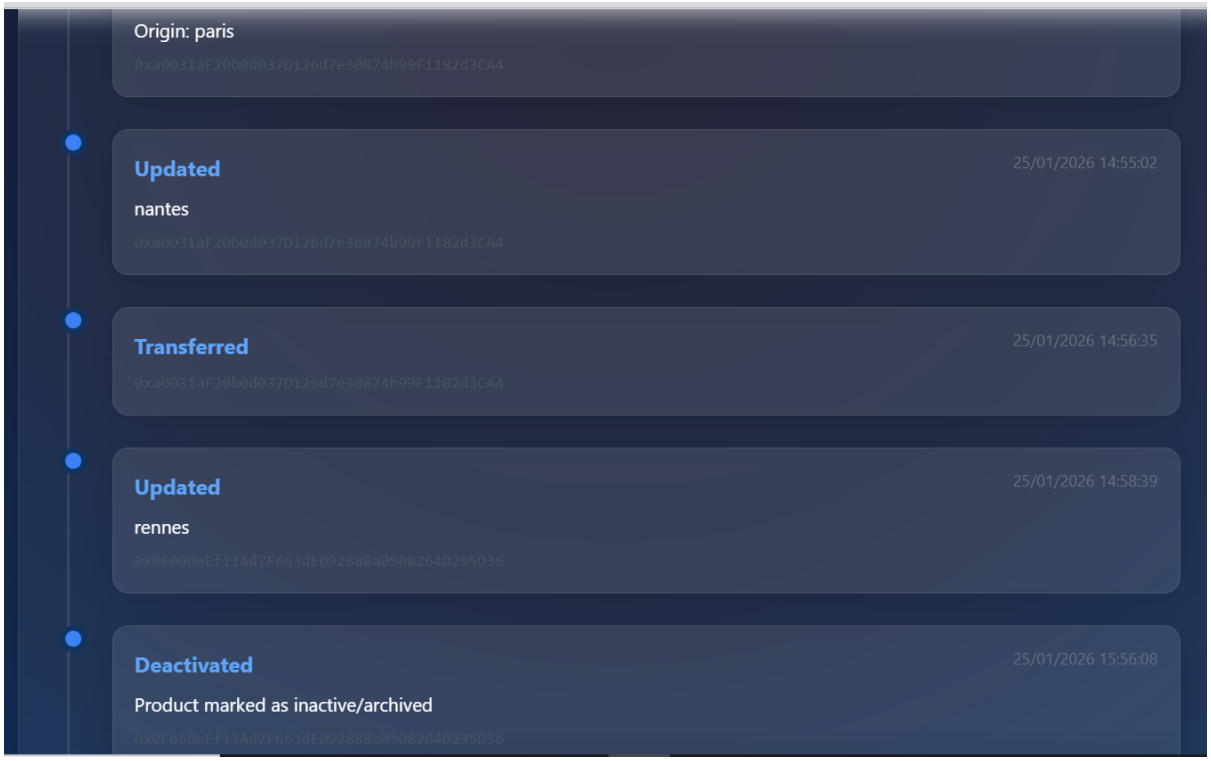
Stakeholders must sign a final transaction to archive the product, providing a cryptographic lock on its historical record.



9.4.2 Archival Confirmation

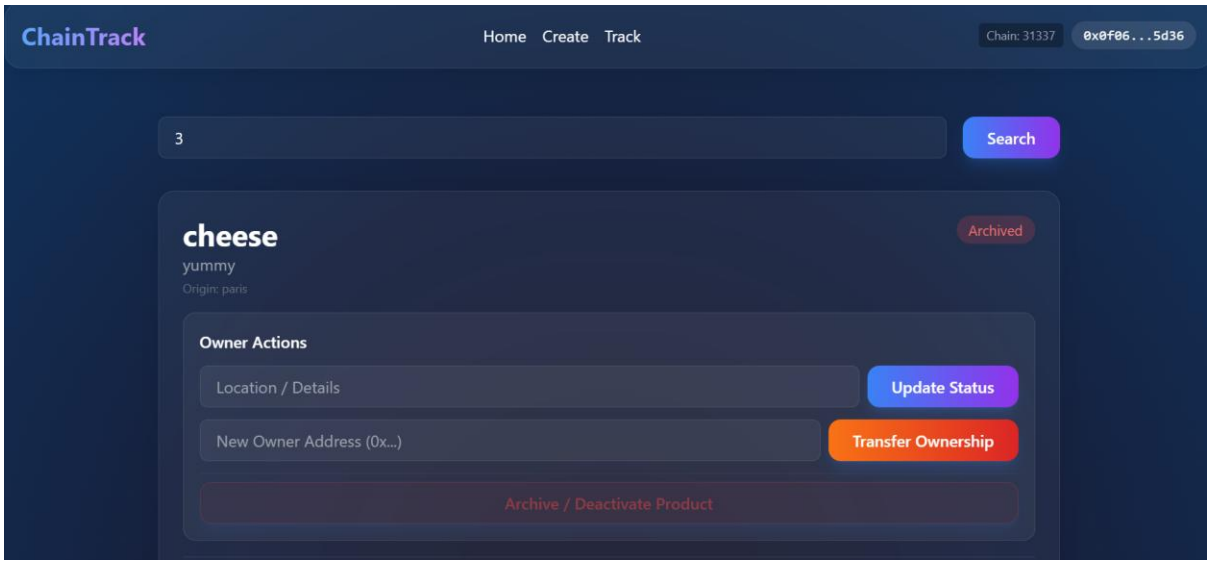
A final system notification confirms that the product has been successfully decommissioned.





9.4.3 Final State Visualization

The UI displays a clear "Archived" status badge, signaling to all viewers that the item's tracked journey is complete.



10 Case Studies and Potential Applications

- **Luxury Goods**: Protecting high-value items like watches or designer bags from the resale of sophisticated fakes.
- **Agriculture**: Tracking "Farm-to-Fork" for food safety, enabling targeted recalls if a specific batch is found to be contaminated.

- **Pharmaceuticals**: Ensuring that medicines are genuine and have been stored correctly during transport.

11 The IoB Perspective: Future Integrations

11.1 IoT and Autonomous Sensor Feedback

The next frontier for ChainTrack is the automated logging of data via IoT sensors. For example, a temperature sensor in a cold-chain truck could automatically call the ***updateProduct*** function if the temperature exceeds a threshold. This removes human error and provides objective, real-time proof of handling quality.

11.2 Zero-Knowledge Proofs (ZK-Proofs)

For enterprise supply chains, privacy is often a concern. Competitors shouldn't necessarily see each other's suppliers. ZK-Proofs like *zk-SNARKs* could allow a manufacturer to prove a product is "Organic" or "Ethically Sourced" without revealing the specific supplier or farm location.

11.3 Cross-Chain Interoperability

Future versions could leverage protocols like Polkadot or Cosmos to allow the SupplyChain contract to interact with other blockchains, such as specialized logistics chains or global financial networks.

12 Socio-Economic and Environmental Impact

12.1 Transparency and Ethical Sourcing

By providing a verifiable path of origin, consumers can support brands that adhere to fair labor practices and environmentally friendly sourcing. This creates a market incentive for sustainability.

12.2 Reducing Administrative Overhead

By automating the verification process, companies can significantly reduce the amount of paperwork and manual auditing required, leading to leaner and more efficient operations.

12.3 Sustainability and Carbon Tracking

Blockchain can be used to track the carbon footprint of a product as it moves through the supply chain, providing a transparent record for institutional reporting and consumer awareness.

13 Quick Testing via Remix

For a zero-installation verification of the smart contract logic, the SupplyChain.sol contract can be tested directly in the Remix IDE (remix.ethereum.org).

Steps to Test:

1. Load Code: Copy the contents of backend/contracts/SupplyChain.sol into a new file in Remix.
2. Compile: Use the Solidity Compiler (version 0.8.20 or higher) and click "Compile SupplyChain.sol".

3. Deploy: Go to the "Deploy & Run Transactions" tab, select "Remix VM" as the environment, and click Deploy.
4. Interact:

Register a Product: Expand the deployed contract and call createProduct with strings like "Apple", "Organic", "France".

Verify History: Call getProductHistory with ID 1 to see the initial "Created" record.

Test Transfer: Copy a different address from the Remix Account list, and call transferProduct with ID 1 and that address.

Security Check: Switch to a different account and try to call updateProduct on ID 1. The transaction should fail, demonstrating the built-in access control logic.

Conclusion

This report presented ChainTrack, a blockchain-based technical framework designed to address fundamental limitations of traditional supply chain systems, particularly in terms of transparency, traceability, and trust. By leveraging Ethereum-compatible smart contracts within the Internet of Blockchain (IoB) paradigm, ChainTrack demonstrates how physical goods can be represented as immutable digital twins whose lifecycle events are securely recorded and verifiable by all stakeholders without reliance on a centralized authority.

From a technical standpoint, the project illustrates the practical integration of decentralized ledger technology, programmable smart contracts, and modern web applications into a coherent end-to-end system. The smart contract layer enforces strict ownership and state transition rules, while the cumulative metadata model ensures complete auditability and resistance to data tampering. The frontend orchestration further highlights how Web3 applications can remain usable and intuitive despite the inherent asynchronicity and complexity of blockchain interactions.

Beyond its technical contributions, ChainTrack emphasizes the broader socio-economic and environmental implications of decentralized provenance systems. Enhanced transparency can help combat counterfeiting, encourage ethical sourcing practices, and reduce administrative overhead across supply chains. Moreover, the ability to trace product history in a verifiable manner opens new opportunities for sustainability reporting and regulatory compliance, particularly in industries where accountability is critical.

Nevertheless, this work also acknowledges existing challenges that must be addressed for real-world deployment, including scalability constraints, privacy concerns, and adoption barriers among heterogeneous stakeholders. While ChainTrack is presented as a prototype rather than a production-ready solution, it provides a robust and extensible foundation for future research. Potential extensions include IoT-driven automation, privacy-preserving verification through zero-knowledge proofs, and cross-chain interoperability.

In conclusion, ChainTrack demonstrates that blockchain-based trust infrastructures are not merely theoretical constructs, but practical tools capable of reshaping how global supply chains are designed, monitored, and governed. The architecture and concepts explored in this project contribute to ongoing research on decentralized systems and highlight the growing role of blockchain as a foundational technology for transparent and accountable digital ecosystems.

References

1. Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System.
2. Wood, G. (2014). Ethereum: A Secure Decentralised Generalised Transaction Ledger.
3. Tapscott, D., & Tapscott, A. (2016). Blockchain Revolution.
4. Buterin, V. (2013). Ethereum White Paper.
5. IOB Course Materials: "Foundations of Decentralized Trust."

Annex A – Smart Contract Implementation (Solidity)

This annex presents the Solidity smart contract used in the ChainTrack system.

```
contract SupplyChain {
    struct History {
        string action; // "Created", "Transferred", "Updated"
        string details; // Location or handling info
        address actor;
        uint256 timestamp;
    }

    struct Product {
        uint256 id;
        string name;
        string description;
        string origin;
        address currentOwner;
        bool isActive;
        History[] history;
    }

    uint256 private _productIds;
    mapping(uint256 => Product) public products;

    event ProductCreated(uint256 indexed id, string name, address indexed owner);
    event ProductTransferred(uint256 indexed id, address indexed from, address
indexed to, string details);
    event ProductUpdated(uint256 indexed id, string details, address indexed actor);
    event ProductDeactivated(uint256 indexed id, address indexed actor);

    function createProduct(string memory _name, string memory _description, string
memory _origin) public returns (uint256) {
        _productIds++;
        uint256 newId = _productIds;

        Product storage newProduct = products[newId];
        newProduct.id = newId;
        newProduct.name = _name;
        newProduct.description = _description;
        newProduct.origin = _origin;
        newProduct.currentOwner = msg.sender;
        newProduct.isActive = true;

        newProduct.history.push(History({
            action: "Created",
            details: string(abi.encodePacked("Origin: ", _origin)),
            actor: msg.sender,
            timestamp: block.timestamp
        }));
```

```
        emit ProductCreated(newId, _name, msg.sender);
        return newId;
    }

    function transferProduct(uint256 _productId, address _newOwner, string memory
_details) public {
        Product storage product = products[_productId];
        require(product.isActive, "Product is not active");
        require(msg.sender == product.currentOwner, "Only owner can transfer");
        require(_newOwner != address(0), "Invalid new owner");

        address previousOwner = product.currentOwner;
        product.currentOwner = _newOwner;

        product.history.push(History({
            action: "Transferred",
            details: _details,
            actor: msg.sender,
            timestamp: block.timestamp
        }));

        emit ProductTransferred(_productId, previousOwner, _newOwner, _details);
    }

    function updateProduct(uint256 _productId, string memory _details) public {
        Product storage product = products[_productId];
        require(product.isActive, "Product is not active");
        require(msg.sender == product.currentOwner, "Only owner can update");

        product.history.push(History({
            action: "Updated",
            details: _details,
            actor: msg.sender,
            timestamp: block.timestamp
        }));

        emit ProductUpdated(_productId, _details, msg.sender);
    }

    function deactivateProduct(uint256 _productId) public {
        Product storage product = products[_productId];
        require(product.isActive, "Product is already inactive");
        require(msg.sender == product.currentOwner, "Only owner can deactivate");

        product.isActive = false;

        product.history.push(History({
            action: "Deactivated",
            details: "Product marked as inactive/archived",
            actor: msg.sender,
            timestamp: block.timestamp
```

```
        }));

        emit ProductDeactivated(_productId, msg.sender);
    }

    function getProduct(uint256 _productId) public view returns (
        uint256 id,
        string memory name,
        string memory description,
        string memory origin,
        address currentOwner,
        bool isActive
    ) {
        require(_productId > 0 && _productId <= _productIds, "Product does not
exist");
        Product storage product = products[_productId];
        return (
            product.id,
            product.name,
            product.description,
            product.origin,
            product.currentOwner,
            product.isActive
        );
    }

    function getProductHistory(uint256 _productId) public view returns (History[]
memory) {
        require(_productId > 0 && _productId <= _productIds, "Product does not
exist");
        return products[_productId].history;
    }
}
```